

Topic: Keras Tuner | Hyperparameter Tuning a Neural Network

1. What Are Hyperparameters?

Before diving into Keras Tuner, let's understand what **hyperparameters** are.

- **Parameters** are learned by the model during training (like weights and biases).
- **Hyperparameters** are set *before* training — they control how the model learns.

Examples of hyperparameters:

- Learning rate
- Number of layers
- Number of neurons per layer
- Batch size
- Number of epochs
- Type of optimizer (Adam, SGD, etc.)
- Dropout rate

The **goal** of hyperparameter tuning is to find the *best combination* of these values to get the highest accuracy and lowest loss.

2. Why Do We Need Hyperparameter Tuning?

Because:

- If the learning rate is too **high**, the model may diverge (won't learn).
- If it's too **low**, training becomes too slow.
- Too many neurons/layers = **overfitting**.
- Too few neurons/layers = **underfitting**.

So, choosing the right hyperparameters makes training **efficient** and the model **generalizes well**.

3. What Is Keras Tuner?

KerasTuner is a library by TensorFlow/Keras that helps **automate the search** for the best hyperparameters.

Instead of manually guessing values (like trying 0.01 or 0.001 for learning rate), KerasTuner **tests different combinations automatically** and tells you which one performs best.

➔ Think of it as a *smart assistant* for finding your model's best configuration.

4. How Does Keras Tuner Work?

You define:

1. **The model structure** (as a function).
2. **The search space**, what hyperparameters you want to tune and their possible ranges.
3. **The objective**, what metric to optimize (like accuracy or validation loss).

Then, KerasTuner automatically:

- Trains the model with different combinations,
- Evaluates the results, and
- Suggests the **best hyperparameters**.

5. Keras Tuner Workflow (Steps)

Step 1: Install and Import

```
!pip install keras-tuner
from kerastuner import RandomSearch
```

Step 2: Define a Model-Building Function

```
def build_model(hp):
    model = keras.Sequential()
    model.add(keras.layers.Dense(
        units=hp.Int('units', min_value=32, max_value=512, step=32),
        activation='relu'
    ))
    model.add(keras.layers.Dense(10, activation='softmax'))
    model.compile(
        optimizer=keras.optimizers.Adam(
            hp.Choice('learning_rate', [1e-2, 1e-3, 1e-4])
        ),
        loss='sparse_categorical_crossentropy',
        metrics=['accuracy']
    )
    return model
```

Step 3: Choose a Tuner

KerasTuner provides multiple search algorithms:

1. **Random Search** – tries random combinations (fast and simple).
2. **Hyperband** – trains quickly using adaptive resource allocation.
3. **Bayesian Optimization** – intelligently chooses new hyperparameters based on past performance.

Example:

```
tuner = RandomSearch(
```

```
build_model,  
objective='val_accuracy',  
max_trials=5,  
directory='my_dir',  
project_name='intro_to_kt'  
)
```

Step 4: Run the Search

```
tuner.search(x_train, y_train, epochs=10, validation_data=(x_val, y_val))
```

Step 5: Get the Best Results

```
best_hps = tuner.get_best_hyperparameters(num_trials=1)[0]
```

Step 6: Train the Best Model

```
model = tuner.hypermodel.build(best_hps)  
model.fit(x_train, y_train, epochs=20, validation_data=(x_val, y_val))
```

6. Main Algorithms in KerasTuner

Algorithm	Idea	Best For
Random Search	Randomly picks values	Quick baseline search
Hyperband	Uses early stopping, drops bad trials fast	Large search space
Bayesian Optimization	Learns from previous trials to pick better next ones	Small to medium search space; more precise

7. Advantages of KerasTuner

- Automates the tuning process
- Works well with Keras models
- Easy to integrate and extend
- Saves time by skipping poor-performing combinations
- Helps achieve better model performance

8. Common Mistakes to Avoid

- Too large search space — makes tuning slow
- Too few trials — may miss the best combination
- Not using validation data — can lead to overfitting
- Forgetting to save the best model — you'll lose progress if code restarts

9. Summary (Simple View)

Step	What Happens
1	Define what hyperparameters to tune
2	Choose tuner (Random, Hyperband, Bayesian)
3	Run search to test multiple models
4	Find best combination
5	Retrain using best settings

In Simple Words:

“KerasTuner is like a smart helper that experiments with different learning settings automatically, finds the best combination for your model, and saves you from wasting time guessing hyperparameters manually.”
